

lambda expression variations, for a single parameter

This slide shows the different ways to declare a single parameter in a lambda expression.

Lambda Expression	Description
<code>element -> System.out.println(element);</code>	A single parameter without a type can omit the parentheses.
<code>(element) -> System.out.println(element);</code>	Parentheses are optional.
<code>(String element) -> System.out.println(element);</code>	Parentheses required if a reference type is specified.
<code>(var element) -> System.out.println(element);</code>	A reference type can be var.

lambda expression variations, the lambda body

The lambda body can be a single expression or can contain a lambda body in opening and closing curly braces.

Lambda Expression	Description
<pre>(element) -> System.out.println(element);</pre>	An expression can be a single statement. Like a switch expression, that does not require yield for a single statement result, the use of return is not needed and would result in a compiler error.
<pre>(var element) -> { char first = element.charAt(0); System.out.println(element + " means " + first); };</pre>	An expression can be a code block. Like a switch expression, that requires yield, a lambda that returns a value, would require a final return statement. All statements in the block must end with semi-colons.

Lambda expressions with multiple parameters

The rules for multiple parameters used in a lambda expression are shown here.

Lambda Expression	Description
<code>(a, b) -> a + b;</code>	Parentheses are always required. Explicit types are not.
<code>(Integer a, Integer b) -> a + b;</code>	If you use an explicit type for one parameter, you must use explicit types for all the parameters.
<code>(var a, var b) -> a + b;</code>	If you use var for one parameter, you must use var for all parameters.

Lambda expressions that return values

This slide shows the two rules for returning values from a lambda expression.

Lambda Expression	Description
<code>(a, b) -> a + b;</code>	when not using curly braces, the return keyword is <u>unnecessary</u> , and will throw a compiler error.
<code>(a, b) -> { var c = a + b; return c; }</code>	If you use a statement block, meaning you use the curly braces, a return is required.

The Four categories of Functional Interfaces

It's a good idea to know the four basic types of functional interfaces in the [java.util.function](#) package.

There are over forty interfaces in this package.

This slide shows the four categories, with the simplest method shown.

These can all be categorized as one of the following types.

Interface Category	Basic Method Signature	Purpose
Consumer	void accept(T t)	execute code without returning data
Function	R apply(T t)	return a result of an operation or function
Predicate	boolean test(T t)	test if a condition is true or false
Supplier	T get()	return an instance of something

The Consumer interface

On this slide, I'm showing the two most common Consumer interfaces, and the functional method on each.

The Consumer interface takes one argument of any type.

The BiConsumer interface takes two arguments, of two different types.

Interface Name	Method Signature
Consumer	void accept(T t)
BiConsumer	void accept(T t, U u)

A Consumer Lambda Expression Example

This slide shows an example consumer lambda expression. It takes one argument and executes a single statement.

No result is returned.

Example Lambda Expression for Consumer	Consumer Method
<code>s -> System.out.println(s)</code>	<code>void accept(T t)</code>

The Predicate Interface

The predicate interfaces take one or two arguments, and always returns a boolean value. They are used to test a condition, and if the condition is true, to perform an operation.

Interface Name	Method Signature
Predicate	boolean test(T t)
<u>BiPredicate</u>	boolean test(T t, U u)

A Predicate Lambda Expression Example

In this example, the expression takes a String, and tests if it's equal to the literal text "Hello", ignoring case. It returns either true or false.

Example Lambda Expression for Consumer

```
s -> s.equalsIgnoreCase("Hello")
```

The Function interface

On this slide, I'm showing four of the most common interfaces in this category.

Each has a return type, shown here as either T, or R, which stands for result, meaning a result is expected for any of these.

In addition to Function and BiFunction, there is also UnaryOperator and BinaryOperator.

You can think of the UnaryOperator as a Function Interface, but where the argument type is the same as the result type.

Interface Name	Method Signature	Interface Name	Method Signature
Function<T,R>	R apply(T t)	UnaryOperator<T>	T apply(T t)
BiFunction<T,U,R>	R apply(T t, U u)	BinaryOperator<T>	T apply(T t1, T t2)

The Function interface

The BinaryOperator is a BiFunction interface, where both arguments have the same type, as does the result, which is why the result is shown as T, and not R.

This reminds us that the result will be the same type as the arguments to the methods.

I've also included the type parameters with each interface name on this slide, because I wanted you to see that the result for a Function or BiFunction, is declared as the last type argument.

For UnaryOperator and BinaryOperator, there is only one type argument declared, because the types of the arguments and results, will be the same.

Interface Name	Method Signature	Interface Name	Method Signature
<code>Function<T,R></code>	<code>R apply(T t)</code>	<code>UnaryOperator<T></code>	<code>T apply(T t)</code>
<code>BiFunction<T,U,R></code>	<code>R apply(T t, U u)</code>	<code>BinaryOperator<T></code>	<code>T apply(T t1, T t2)</code>

A Function Interface Lambda Expression Example

On this slide, I'm showing an example of a lambda expression which targets a Function interface.

This lambda expression takes a String, s, and splits that String on commas, returning an array of String.

In this case, the argument type, T, is a String, and the result, R, is an array of String.

To demonstrate how to declare a variable of this type, I'm showing a variable declaration as well.

Example Lambda Expression for Function	Function Method	Variable Declaration for this example
<code>s -> s.split(",");</code>	<code>R apply(T t)</code>	<code>Function<String, String[]> f1;</code>

The Supplier Interface

The supplier interface takes no arguments but returns an instance of some type, T.

Interface Name	Method Signature
Supplier	T get()

You can think of this as kind of like a factory method.

It will produce an instance of some object.

However, this doesn't have to be a new or a distinct result returned.

A Supplier Lambda Expression Example

In the example I'm showing you on this slide, I'm using the Random class to generate a random Integer.

This method takes no argument, but lambda expressions can use final or effectively final variables in their expressions, which I'm demonstrating here.

The variable random is an example of a variable from the enclosing code.

Example Lambda Expression for Consumer

```
() -> random.nextInt(1, 100)
```

Valid Lambda Declarations for different number of arguments

This slide shows the many varieties of declaring a parameter type in a lambda expression.

Parentheses are required in all but the one case, where the functional method has a single argument, and you don't specify a type, or use var.

Arguments in Functional Method	Valid lambda syntax
None	<code>() -> statement</code>
One	<code>s -> statement</code> <code>(s) -> statement</code> <code>(var s) -> statement</code> <code>(String s) -> statement</code>
Two • When using var, all arguments must use var. • When specifying explicit types, all arguments must specify explicit types.	<code>(s, t) -> statement</code> <code>(var s, var t) -> statement</code> <code>(String s, List t) -> statement</code>

Valid Lambda Declarations for different number of arguments

When using var as the type, every argument must use var.

When specifying explicit types, every argument must include a specific type.

Arguments in Functional Method	Valid lambda syntax
None	<code>() -> statement</code>
One	<code>s -> statement</code> <code>(s) -> statement</code> <code>(var s) -> statement</code> <code>(String s) -> statement</code>
Two • When using var, all arguments must use var. • When specifying explicit types, all arguments must specify explicit types.	<code>(s, t) -> statement</code> <code>(var s, var t) -> statement</code> <code>(String s, List t) -> statement</code>

Four Types of Method References

This chart shows the four different types of method references, with method reference examples, and a corresponding lambda expression.

Type	Syntax	Method Reference Example	Corresponding Lambda Expression
static method	<code>ClassName::staticMethodName(p1, p2, ... pn)</code>	<code>Integer::sum</code>	<code>(p1, p2) -> p1 + p2</code>
instance method of a particular (Bounded) object	<code>ContainingObject::instanceMethodName(p1, p2, ... pn)</code>	<code>System.out::println</code>	<code>p1 -> System.out.println(p1)</code>
instance method of an arbitrary (Unbounded) object (as determined by p1)	<code>ContainingType[=p1] ::instanceMethodName(p2, ... pn)</code>	<code>String::concat</code>	<code>(p1, p2) -> p1.concat(p2).</code>
constructor	<code>ClassName::new</code>	<code>LPASStudent::new</code>	<code>() -> new LPASStudent()</code>

Method Reference Examples (No arguments and one argument)

This chart shows some of the valid ways to use method references when assigned to different interface types.

These interface types have no arguments in the case of a Supplier, and one argument for the other interfaces.

	No Args	One Argument		
Types of Method References	Supplier	Predicate	Consumer	Function <u>UnaryOperator</u> et. al.
Reference Type (Static)				
Reference Type (Constructor)	Employee::new		n/a	Employee::new
Bounded Receiver (Instance)			System.out::println	
Unbounded Receiver (Instance)	n/a	String::isEmpty	List::clear	String::length

Method Reference Examples (No arguments and one argument)

If a cell is empty, it's not because it's not valid, but because there are many possibilities.

n/a means not applicable, so a Supplier or an interface method that has no arguments, can never be a target for the unbounded receiver type of method reference.

	No Args	One Argument		
Types of Method References	Supplier	Predicate	Consumer	Function <u>UnaryOperator</u> et. al.
Reference Type (Static)				
Reference Type (Constructor)	Employee::new		n/a	Employee::new
Bounded Receiver (Instance)			System.out::println	
Unbounded Receiver (Instance)	n/a	String::isEmpty	List::clear	String::length

Method Reference Examples (Two Arguments)

This chart shows some of the valid ways to use method references when assigned to different interface types.

These interface types have two arguments, and therefore it's more common to see the unbounded receiver method references used for these.

	Two Arguments		
Types of Method References	<u>BiPredicate</u>	<u>BiConsumer</u>	<u>BiFunction</u> <u>BinaryOperator</u> et. al.
Reference Type (Static)			Integer::sum
Reference Type (Constructor)			Employee:: new
Bounded Receiver (Instance)		System.out::printf	new Random()::nextInt
Unbounded Receiver (Instance)	String::equals	List::add	String::concat String::split

Convenience Methods on functional interfaces in java.util.function package

Category of Interface	Convenience method example	Notes
Function	function1. andThen (function2)	Not implemented on <u>IntFunction</u> , <u>DoubleFunction</u> , <u>LongFunction</u>
Function	function2. compose (function1)	Only implemented on <u>Function</u> & <u>UnaryOperator</u>
Consumer	consumer1. andThen (consumer2)	
Predicate	predicate1. and (predicate2)	
Predicate	predicate1. or (predicate2)	
Predicate	predicate1. negate ()	

For andThen, and compose, on the Function category of interfaces, any Interim functions are not required to have the same type arguments.

Instead, one function's output becomes the next function's input, and the next function's output is not constrained to any specific type, except the last function executed in the chain.

Convenience Methods on functional interfaces in java.util.function package

Category of Interface	Convenience method example	Notes
Function	function1. andThen (function2)	Not implemented on <u>IntFunction</u> , <u>DoubleFunction</u> , <u>LongFunction</u>
Function	function2. compose (function1)	Only implemented on <u>Function</u> & <u>UnaryOperator</u>
Consumer	consumer1. andThen (consumer2)	
Predicate	predicate1. and (predicate2)	
Predicate	predicate1. or (predicate2)	
Predicate	predicate1. negate ()	

The Consumer's andThen method is different, because it never returns a result, so you use this when you're chaining methods independent of one another.

The Predicate methods always return a boolean, which will combine the output of the two expressions, to obtain a final boolean result.

Comparator's additional helper methods

I want to cover the additional convenience methods now, since, as you can see from this table, many take a functional interface instance, as an argument.

Type of Method	Method Signature
static	Comparator comparing (Function <u>keyExtractor</u>)
static	Comparator <u>naturalOrder</u> ()
static	Comparator <u>reverseOrder</u> ()
default	Comparator <u>thenComparing</u> (Comparator other)
default	Comparator <u>thenComparing</u> (Function <u>keyExtractor</u>)
default	Comparator reversed ()

There is a comparing static method, and an overloaded default method named thenComparing, and finally a default reversed method.